Experiment 2B: Parallel Merge Sort using OpenMP

This guide explains how to run Parallel Merge Sort using OpenMP on both Windows and Linux/Ubuntu systems. It includes corrected code, execution steps, example output, and viva questions for beginners.

Problem Statement: Write a program to implement Parallel Merge Sort using OpenMP and measure execution time.

Correct Program Code

```
#include <iostream>
#include <omp.h>

using namespace std;

// Merge Function
void merge(int arr[], int left, int mid, int right)
{
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int* L = new int[n1];
    int* R = new int[n2];

    // Copy data
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];

    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0;
    int j = 0;
    int k = left;

    // Merge arrays
    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
        {
            arr[k] = L[i];
            i++;
        }
        else
        {
            arr[k] = R[j];
            j++;
        }

        k++;
    }

    // Remaining elements
    while (i < n1)
    {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2)
    {
        arr[k] = R[j];
        j++;
        k++;
    }

    delete[] L;
    delete[] R;
}
// Parallel Merge Sort
```

```
void parallelMergeSort(int arr[], int left, int right)
{
    if (left < right)
    {
        int mid = (left + right) / 2;

        #pragma omp parallel sections
        {
            #pragma omp section
            parallelMergeSort(arr, left, mid);

            #pragma omp section
            parallelMergeSort(arr, mid + 1, right);
        }

        merge(arr, left, mid, right);
    }
}

int main()
{
    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    int* arr = new int[n];

    cout << "Enter elements:\n";

    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }

    double start_time = omp_get_wtime();

    parallelMergeSort(arr, 0, n - 1);

    double end_time = omp_get_wtime();

    cout << "\nSorted Array:\n";

    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }

    cout << "\n\nExecution Time: "
         << (end_time - start_time) * 1000
         << " milliseconds\n";

    delete[] arr;

    return 0;
}
```

How to Run on Windows

1. Install Code::Blocks with MinGW compiler.
2. Open Code::Blocks.
3. Go to File → New → Empty File.
4. Paste the program code.
5. Save file as merge_sort.cpp.
6. Go to Settings → Compiler.
7. Open Other Compiler Options.
8. Add: -fopenmp
9. Click OK.

10. Press F9 or click Build and Run.

How to Run on Linux / Ubuntu

Install required software:

```
sudo apt update
sudo apt install g++
sudo apt install libomp-dev
```

Compile and run:

```
g++ -fopenmp merge_sort.cpp -o merge_sort

./merge_sort
```

Sample Input

```
Enter number of elements: 5

Enter elements:
9
2
7
1
5
```

Expected Output

```
Sorted Array:
1 2 5 7 9

Execution Time: 35 milliseconds
```

Important Viva Questions

Question	Answer
Which algorithm is used?	Merge Sort using divide-and-conquer.
Why use OpenMP?	To sort left and right halves simultaneously.
Which OpenMP directive is used?	#pragma omp parallel sections
What is time complexity?	O(n log n)